Asolaria System Card for OpenAI Board Review (Cited)

Date: 2026-03-09

Scope: board-critical claims only, with inline evidence references

Companion artifact: `reports/asolaria-openai-board-system-card-20260309.md` for the fuller narrative and exhaustive inventories

Sensitivity: sanitized, no tokens or secrets

1. Executive thesis

Asolaria is a local-first agent control plane, not a chat wrapper and not a single-agent shell. Its core shape is already visible in local code and runbooks: multi-provider reasoning, policy-gated tool invocation, async task routing, persistent memory and mistake capture, graph-native runtime telemetry, desktop and phone actuation, and relay-based multi-machine coordination. [Sources: `src/brainOrchestrator.js`; `src/jobQueue.js`; `src/graphRuntimeStore.js`; `docs/GATEWAY_TOOL_AUTHORITY.md`; `docs/AUTHORITY_MODES.md`; `codex-bridge/bridge.js`; `codex-bridge/watchdog.ps1`]

This matters to OpenAI because Asolaria is already converging on the real environment power users want around Codex and Symphony: governed execution, durable workflow lanes, compact integration fabric, and operator-grade recovery instead of "prompt and pray." [Sources: `docs/SYMPHONY_INTEGRATION.md`; `docs/AUGMENT_CONTEXT_SIDECAR.md`; `src/connectors/workerRouter.js`; `src/connectors/voiceConnector.js`]

2. System shape

The source tree and startup docs show a layered system rather than a single runtime blob:

- interaction surfaces for app, mobile, approvals, and phone-safe views
- execution surfaces for queueing, routing, local CLI work, and connector dispatch
- integration surfaces for Google, GCP, Vertex, GitHub, Slack, Telegram, Microsoft, Atlassian, NotebookLM, Symphony, and Augment context
- governance surfaces for authority modes, approval gates, readiness drills, and audit verification
- ops surfaces for watchdogs, tunnels, startup contracts, and recovery runbooks

[Sources: `README.md`; `docs/CANONICAL_SOURCE.md`; `docs/STARTUP_CONTRACT_V1.md`; `docs/GATEWAY_TOOL_AUTHORITY.md`; `src/brainOrchestrator.js`; `src/connectors/workerRouter.js`; `codex-bridge/ASOLARIA_COLONY_STARTUP.md`]

The startup contract is especially revealing. It defines a role model with `Asolaria Supervisor`, `Vertex`, `Rook`, and `Falcon`, requires identity handshakes before task dispatch, and establishes the phone path before the phone worker is trusted. That is systems thinking, not demo logic. [Sources: `docs/STARTUP_CONTRACT_V1.md`; `codex-bridge/ASOLARIA_COLONY_STARTUP.md`]

3. Governed execution and API posture

The gateway is one of the strongest parts of the system. It exposes explicit control surfaces for `/invoke`, `/approvals`, `/tools/policy`, `/authority/status`, `/authority/readiness`, `/authority/mode/request`, `/authority/mode/confirm`, `/authority/mode/rollback`, and `/audit/*`. Approval records are one-time use, tied to tool id plus input hash, and expire by TTL. Audit integrity can block control writes if verification fails. [Sources: `docs/GATEWAY_TOOL_AUTHORITY.md`; `docs/AUTHORITY_MODES.md`]

Authority is not binary. The gateway supports `operator_primary`, `shared_control`, and `asolaria_primary`, and transitions to autonomous control are two-step, rollbackable, and guardable by heartbeat and audit readiness checks. [Sources: `docs/AUTHORITY_MODES.md`]

This is why the system feels credible. Tool use is treated as a governed operating surface instead of an invisible side effect of reasoning. [Sources: `docs/GATEWAY_TOOL_AUTHORITY.md`; `docs/GATEWAY_AUDIT_LOG.md`]

4. Runtime footprint and inventories

The current footprint is already large enough to matter:

- 40 connector modules in `src/connectors`
- 53 indexed skills
- 32 indexed tool actions
- 5 tracked project scopes in the project store

[Sources: direct local count of `src/connectors`; `reports/skills-tools-mistakes-index-20260305-015506.json`; `data/projects.json`]

The authoritative connector inventory includes `augmentContextConnector`, `desktopControlConnector`, `gcpConnector`, `githubConnector`, `notebooklmConnector`, `phoneBrowserHistoryConnector`, `phoneDeveloperOptionsConnector`, `phoneMirrorConnector`, `symphonyConnector`, `vertexConnector`, `voiceConnector`, `wolConnector`, and the `workerRouter`, among others. [Sources: `src/connectors/*`; `reports/asolaria-openai-board-system-card-20260309.md`]

The indexed skill inventory spans operational work, browser and phone workflows, mistake avoidance, visual audit, Google and Vertex status, codex skill references, and multiple imported Codex skill references such as `codex.ref.openai-docs`, `codex.ref.linear`, `codex.ref.playwright`, `codex.ref.pdf`, and `codex.ref.google-code-wiki`. [Sources: `reports/skills-tools-mistakes-index-20260305-015506.json`]

The indexed tool-action inventory includes status, rebuild, Google and GCP checks, Vertex budget status, capture actions, mobile browser history checks, Chrome profile listing, URL open flows, and WebMCP inspection/task surfaces. [Sources: `reports/skills-tools-mistakes-index-20260305-015506.json`]

5. OpenAI-native operating lanes

Codex is not merely present as a model endpoint. It is in the default reasoning chain and also appears as the highest-trust local execution lane. The orchestrator's provider chain starts with `codex`, and the worker

router explicitly models `local_codex` as the privileged local engineering lane. [Sources: `src/brainOrchestrator.js`; `src/connectors/workerRouter.js`]

Symphony is integrated as a managed sidecar rather than a loose add-on. Asolaria stores repo root, workflow path, Linear project slug, Codex command, and optional custom launch details. It exposes status, config, start, stop, and restart endpoints, performs prerequisite checks, and handles the Windows-specific startup path through `mise exec -- escript ./bin/symphony`. [Sources: `docs/SYMPHONY_INTEGRATION.md`; `src/connectors/symphonyConnector.js`]

Linear appears as part of the Symphony execution semantics, not as a disconnected ticket feed. The system explicitly binds workflow execution to a `linearProjectSlug`, issue pickup, and long-running handoff behavior. [Sources: `docs/SYMPHONY_INTEGRATION.md`; `src/connectors/symphonyConnector.js`; `src/connectors/workerRouter.js`]

The voice lane is also directly relevant to OpenAI. The local voice connector defaults its cloud transcription model to `gpt-4o-mini-transcribe`, while maintaining fallbacks and adjacent live-audio paths. [Sources: `src/connectors/voiceConnector.js`]

6. Auggie / Augment context

Augment is integrated in a disciplined way: as a read-only context sidecar, not as a second uncontrolled brain. The integration supports `local_cli` for same-machine workspace retrieval and `remote_mcp` for a narrow remote bridge. Allowed tools are restricted to `codebase-retrieval`, and generated provider patches are always `read_only` with `never` approval. [Sources: `docs/AUGMENT_CONTEXT_SIDECAR.md`; `src/connectors/augmentContextConnector.js`]

This is important because it shows architectural restraint. Asolaria uses external context systems to strengthen local reasoning, not to multiply opaque autonomous actors. [Sources: `docs/AUGMENT_CONTEXT_SIDECAR.md`; `src/connectors/externalBrainConnector.js`]

7. Google, GCP, Vertex, and NotebookLM

Google and GCP are first-class system surfaces. The GCP connector is service-account based, defaults to read-only `cloud-platform.read-only`, enforces allowed projects, and separately allowlists the admin services that can be enabled. [Sources: `src/connectors/gcpConnector.js`]

Vertex is similarly structured. It has explicit config for project, location, and model, plus a daily budget system with request ceilings, prompt-token ceilings, output-token ceilings, warning thresholds, and usage tracking. Budget overruns are enforced in code. [Sources: `src/connectors/vertexConnector.js`]

NotebookLM is not theoretical either. The current build profile records Google, GCP, and Vertex as enabled and configured, and notes NotebookLM connector presence. [Sources: `reports/asolaria-current-build-profile.md`; `src/connectors/notebooklmConnector.js`]

The larger board point is that Asolaria already treats cloud cognition and workspace systems as governed operating substrates, not just API experiments. [Sources: `src/connectors/gcpConnector.js`; `src/connectors/vertexConnector.js`; `src/connectors/notebooklmConnector.js`]

8. Desktop, browser, phone, and operator assistance abilities

Asolaria crosses abstraction levels cleanly. The connector layer contains desktop capture, desktop control, display, screenshot, Chrome, phone mirror, phone browser history, phone developer options, pro mirror, voice, and wake-on-LAN surfaces. [Sources: `src/connectors/*`; `reports/asolaria-openai-board-system-card-20260309.md`]

The phone lane is especially notable. The startup contract requires phone path establishment before `Falcon` is trusted. The phone runbooks cover USB debugging, ADB health checks, recovery handling, and controlled sideload procedures. The phone-related connectors explicitly reference ADB paths and device interaction logic. [Sources: `docs/STARTUP_CONTRACT_V1.md`; `tools/Phone-Recovery-Runbook.md`; `src/connectors/phoneMirrorConnector.js`; `src/connectors/phoneBrowserHistoryConnector.js`; `src/connectors/phoneDeveloperOptionsConnector.js`; `src/connectors/phoneProMirrorConnector.js`]

There is also evidence of AnyDesk-assisted operator flows in the wider operating environment, including a dedicated `anydesk-sentry` monitor path inside `codex-bridge`. [Sources: `codex-bridge/anydesk-sentry/anydesk-sentry.js`; `reports/asolaria-openai-board-system-card-20260309.md`]

This is why the system reads as an operating environment rather than an app. It already spans reasoning, actuation, and recovery across desktop and mobile contexts. [Sources: `src/connectors/*`; `tools/Phone-Recovery-Runbook.md`; `codex-bridge/anydesk-sentry/anydesk-sentry.js`]

9. Projects, memory, and mistakes

The project store is a real built-in operating primitive. Current tracked scopes include `Asolaria Core`, `AI Healthcare`, `Bridge Ops`, `Skills Lab`, and `Task Wave 20260222144718 Updated`, grouped across `projects`, `tasks`, `skills`, and `runtime`. [Sources: `data/projects.json`; `src/projectStore.js`]

The memory store persists turns, snapshots memory over time, and includes compaction-oriented logic instead of treating each session as disposable. [Sources: `src/memoryStore.js`]

The mistake system is one of the most differentiated features. The ledger writes mistake records into an append-only NDJSON store, sanitizes dangerous context keys, and promotes entries into a mistake-pattern taxonomy. Representative patterns in the raw store include `swarm_mode_toggle`, `stale_agent_id`, `agent_colony_idle_escalation`, `shell_quoting`, `missing_dependency`, `tool_fallback`, and `sweep_failure`. [Sources: `src/mistakeLedgerStore.js`; `src/mistakePatternStore.js`; local mistake-pattern store snapshot summarized in `reports/asolaria-openai-board-system-card-20260309.md`]

This means Asolaria is not just logging errors. It is turning failure into reusable operational memory. That is an unusually mature pattern for an agentic system. [Sources: `src/mistakeLedgerStore.js`; `src/mistakePatternStore.js`]

10. Graph runtime and the GNN integration story

The graph runtime is live today. The graph store records typed `graph_event` and `action_manifest` records with actor, subject, target, context, policy, detail, status, and computed risk. [Sources: `src/graphRuntimeStore.js`]

The graph export path is live too. `tools/export-graph-runtime-dataset.js` builds training datasets over graph-runtime windows and writes them into `reports/graph-runtime-datasets`. [Sources: `tools/export-graph-runtime-dataset.js`]

The key architectural move is how the GNN idea entered the system. Internal review did not treat external graph papers as a production blueprint. Instead, a healthcare anomaly-detection codebase and external graph research were decomposed into reusable parts, and Asolaria's own control plane was identified as the natural home for a typed temporal heterograph focused on edge-risk scoring and anomaly support. [Sources: `reports/graph-runtime-research-review-20260307.md`; `reports/asolaria-system-card-20260303.md`; `tools/export-graph-runtime-dataset.js`]

The honest status matters: the graph substrate is live, the dataset-export path is live, and the sidecar design is live, but there is no evidence that GNN inference currently sits in the critical control path. That is the correct design. Policy and approvals remain the root of trust, while graph learning is positioned as a future ranking and anomaly sidecar. [Sources: `reports/graph-runtime-research-review-20260307.md`; `src/graphRuntimeStore.js`; `tools/export-graph-runtime-dataset.js`; `tools/Codex-Graph-ControlPlane-Handoff.prompt.md`]

That is the grown-up version of "state of the art" here: instrument first, constrain the model, keep governance authoritative, and only then move graph learning closer to live control. [Sources: `reports/graph-runtime-research-review-20260307.md`; `src/graphRuntimeStore.js`]

11. Relay colony and cross-machine bootstrap

Asolaria's relay layer is real infrastructure. The bridge server supports room-based messaging, bearer auth, optional on-disk persistence, room policy files, allowlists, and per-sender token enforcement. [Sources: `codex-bridge/bridge.js`; `codex-bridge/AIRLOCK_BOTS.md`]

The watchdog is a reliability layer that keeps relay, tunnel, and bot processes alive, and the colony startup contract defines permanent worker names, startup order, identity verification, and kill-on-mismatch behavior. [Sources: `codex-bridge/watchdog.ps1`; `codex-bridge/ASOLARIA_COLONY_STARTUP.md`]

The bootstrap path is also explicit enough to matter. The friend-computer bootstrap materials define host discovery, toolchain inventory, machine-specific adaptation, verification, and a repeatable route for bringing another workstation into the Codex and Asolaria orbit. That is the relevant evidence behind the "Windows to Mac / friend machine clone" claim: not a single monolithic script, but a documented operational pattern for cross-host build-out using the same control-plane logic. [Sources: `tools/Friend-Codex-Bootstrap-SystemCard.prompt.md`; `tools/Friend-Codex-Bootstrap-TaskPacket.template.md`; `tools/New-Friend-Codex-BridgePack.ps1`; `codex-bridge/AGENT_PROFILE_SPEC.md`]

12. Why this is an OpenAI-relevant system

Asolaria already contains the ingredients OpenAI should care about most:

- Codex as a first-class local execution lane
- Symphony as a durable workflow sidecar
- remote MCP used in compact, policy-aware ways
- authority transitions and approvals as mandatory runtime primitives
- graph-native telemetry instead of opaque agent state
- memory and mistake systems that create longitudinal operational learning
- desktop and phone actuation under documented trust boundaries
- multi-machine expansion through relay and bootstrap patterns

[Sources: `src/brainOrchestrator.js`; `docs/SYMPHONY_INTEGRATION.md`; `docs/AUGMENT_CONTEXT_SIDECAR.md`; `docs/GATEWAY_TOOL_AUTHORITY.md`; `src/graphRuntimeStore.js`; `src/memoryStore.js`; `src/mistakeLedgerStore.js`; `codex-bridge/bridge.js`; `codex-bridge/ASOLARIA_COLONY_STARTUP.md`]

The strategic insight is that Asolaria is already behaving like a frontier-model operating environment, not a single product feature. That is why a board should read it as signal. [Sources: all references above]

13. Honest limits

Several constraints are also visible and should be said plainly:

- there is no evidence that live GNN inference is in the critical control path yet
- there is no single consolidated external-app control matrix that unifies phone, AnyDesk, desktop control, and browser automation under one artifact
- some runtime reports still describe lane-quality gaps and reliability differences across external workers and mobile-related paths

[Sources: `reports/graph-runtime-research-review-20260307.md`; `reports/asolaria-openai-board-system-card-20260309.md`; `reports/asolaria-system-card-20260308-runtime-post-restart.md`]

Those are not disqualifying weaknesses. They are the unfinished edges of a system that is already more operationally serious than most agent stacks. [Sources: `reports/asolaria-openai-board-system-card-20260309.md`]

14. Bottom line

The "wow" is not one connector, one model, or one interface.

The "wow" is that one local-first system already combines governed tool execution, Codex and Symphony lanes, remote MCP discipline, graph-native telemetry, project and memory persistence, mistake learning, desktop and phone agency, and multi-machine expansion inside an auditable operational fabric. [Sources: `src/brainOrchestrator.js`; `src/connectors/workerRouter.js`; `docs/SYMPHONY_INTEGRATION.md`; `docs/AUGMENT_CONTEXT_SIDECAR.md`; `docs/GATEWAY_TOOL_AUTHORITY.md`; `src/graphRuntimeStore.js`; `src/projectStore.js`; `src/memoryStore.js`; `src/mistakeLedgerStore.js`; `codex-bridge/bridge.js`;

codex-bridge/ASOLARIA_COLONY_STARTUP.md]